Bounded Lock-Free Queues: From SPSC to MPMC

Pre-allocated ring buffers · Vyukov sequences · Disruptor pipeline · False-sharing

CS6868 — Project 12

Agenda

01 Motivation & Research Question

02 Ring Buffer Fundamentals

03 SPSC Queue — Lamport's Algorithm

04 MPMC Queue — Vyukov's Algorithm

05 MPSC Queue — One-Sided CAS

06 Disruptor Pipeline (SPSC & MPSC)

07 Testing & Verification

08 Benchmarks & False-Sharing Results

09 Conclusions

Why Bounded Ring-Buffer Queues?

Michael-Scott Queue (Unbounded)

- Allocates one heap node per enqueue
- → GC pressure on hot path
- → Cache misses through allocator free lists
- → Node pointer chasing on dequeue
- Unbounded: memory usage grows until GC
- Performance cliff under allocation pressure

Ring-Buffer Queue (Bounded)

- Pre-allocated fixed array — zero GC per op
- → All data stays hot in L1/L2 cache
- → Pure index arithmetic on enqueue/dequeue
- Bounded: back-pressure when full
- Predictable latency profile
- Eliminates ABA problem on indices

Research Question

How much throughput does a pre-allocated ring buffer gain over the allocation-heavy Michael-Scott queue, and where in the SPSC → MPSC → MPMC design space does the complexity-performance trade-off shift most dramatically?

Design Space

SPSC

1 prod · 1 cons
No CAS needed

→ complexity

MPSC

N prod · 1 cons
CAS on tail only

→ complexity

MPMC

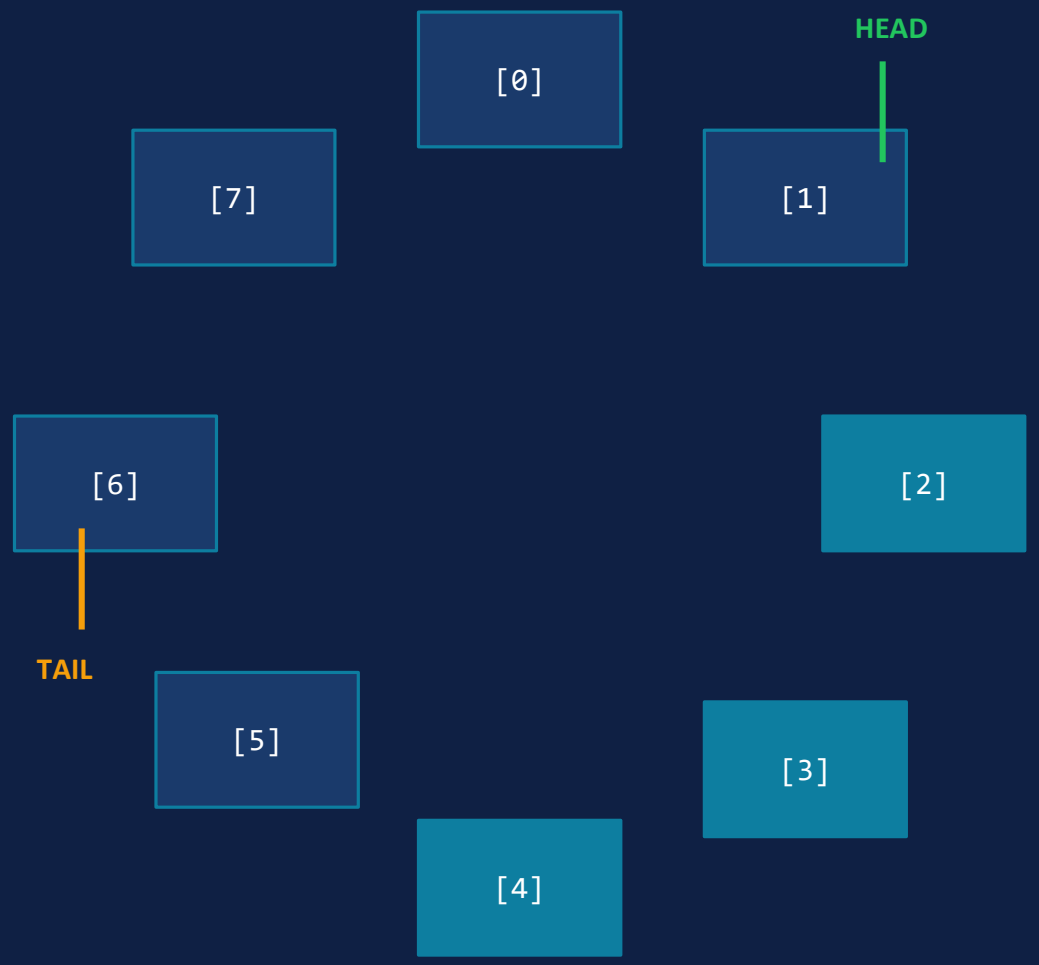
N prod · N cons
CAS on both ends

SECTION 01

Ring Buffer Fundamentals

Ring Buffer: The Core Data Structure

- **Monotonic indices — never wrap**
  - slot = buf[index & (capacity − 1)]
  - No ABA: full ≠ empty by sign of (tail − head)
- **Power-of-two capacity**
  - Cheap modulo: idx & mask instead of idx % capacity
- **Pre-allocated at construction time**
  - Zero allocation on the hot enqueue/dequeue path
- **Back-pressure when full**
  - Producer spin-waits or returns false



Monotonic Indices: Full vs Empty Disambiguation

Key Invariant

```
(* tail - head = number of items in queue *)
tail = head            (* EMPTY: 0 items *)
tail - head = capacity (* FULL:  C items *)

(* Slot access — cheap power-of-2 modulo *)
slot_index = position land mask   (* mask = capacity - 1 *)
```

Example: capacity = 4, enqueue 4, dequeue 2, enqueue 1

Initial	h=0	t=0	·	·	·	·		*empty: tail = head = 0*
Enqueue 4	h=0	t=4	A	B	C	D		*full: tail - head = 4 = capacity*
Dequeue 2	h=2	t=4	·	·	C	D		*partial: tail - head = 2*
Enqueue 1	h=2	t=5	E	·	C	D		*wrap: 5 & 3 = 1 → slot [1]*

SECTION 02

SPSC Queue
Lamport's Algorithm

SPSC: Data Structure

Task 1 *OCaml 5 record type — all fields set once at construction*

```
type 'a t = {
  capacity : int;
  mask     : int;               (* capacity - 1, cheap modulo *)
  buf      : 'a Option.t array; (* pre-allocated ring buffer  *)
  head     : int Atomic.t;      (* consumer writes; producer reads *)
  tail     : int Atomic.t;      (* producer writes; consumer reads *)
}

let make capacity =
  if capacity <= 0 || capacity land (capacity - 1) <> 0 then
    invalid_arg "capacity must be a positive power of 2";
  { capacity;
    mask = capacity - 1;
    buf  = Array.make capacity None;
    head = Atomic.make_contended 0;   (* padded to own cache line *)
    tail = Atomic.make_contended 0 }  (* padded to own cache line *)
```

Atomic.make_contended
Pads each index to its own 64-byte cache line.
Prevents false sharing between producer & consumer cores.

Option.t array
Using None as the empty sentinel lets the GC collect dequeued values promptly.

SPSC: try_enqueue — Producer Side

```
let try_enqueue t v =
  let tl = Atomic.get t.tail in
  let hd = Atomic.get t.head in   (* read consumer progress *)
  if tl - hd >= t.capacity then false  (* FULL — back-pressure *)
  else begin
    t.buf.(tl land t.mask) <- Some v;  (* write value into slot *)
    Atomic.set t.tail (tl + 1);        (* release store *)
    true
  end
```

Step-by-step trace

1. Read tail (producer's own index — no contention)
2. Read head (consumer's index — cross-domain atomic load)
3. Check back-pressure: tail − head ≥ capacity → return false
4. Write value into buf[tail & mask] — plain array store
5. Release-store tail+1 — makes value visible before index advance

NO CAS EVER ISSUED — the producer is sole writer of tail, so plain atomic set suffices. Key performance advantage of SPSC.

SPSC: try_dequeue — Consumer Side

```
let try_dequeue t =
  let hd = Atomic.get t.head in
  let tl = Atomic.get t.tail in   (* read producer progress *)
  if hd >= tl then
    None                           (* EMPTY *)
  else begin
    let v = t.buf.(hd land t.mask) in
    t.buf.(hd land t.mask) <- None; (* clear slot — lets GC collect value *)
    (* Release store: slot clear must complete before head advances *)
    Atomic.set t.head (hd + 1);
    v
  end
```

Memory Ordering Analysis

Operation	Ordering	Why
Atomic.get tail	Acquire	Consumer must see the value written before tail was set
Array write (slot)	Plain	Only one writer; GC barrier handles visibility
Atomic.set tail (producer)	Release	Value in slot must be visible before tail increment
Atomic.set head (consumer)	Release	Slot clear must happen before producer reuses the slot

SPSC: Correctness Invariants

- **Why no CAS is needed**
- Producer is the SOLE writer of tail — no other domain touches it
- Consumer is the SOLE writer of head — no other domain touches it
- Each domain only ever reads the OTHER's index → cross-domain load, not store
- → No two domains ever write the same atomic variable concurrently
- **Why the release store is critical**
- OCaml 5's memory model does NOT guarantee that a value write is visible before the index update unless an ordering is specified
- Atomic.set provides a release fence: all stores before it complete before it
- Consumer's Atomic.get provides an acquire fence: it sees all releases that happened before the matching set
- **ABA impossibility**
- Indices grow monotonically — they never wrap back to 0
- full (tail−head=C) and empty (tail=head) are numerically distinct always

SECTION 03

MPMC Queue
Vyukov's Per-Slot Sequences

MPMC: The Core Idea — Per-Slot Sequence Numbers

Task 2 *Each slot carries its own atomic sequence number encoding its lifecycle state*

```
type 'a slot = {
  sequence : int Atomic.t;  (* lifecycle state for this slot *)
  mutable  value : 'a option;
}

type 'a t = {
  mask        : int;
  buffer      : 'a slot array;
  enqueue_pos : int Atomic.t;  (* global producer cursor — plain Atomic.make *)
  dequeue_pos : int Atomic.t;  (* global consumer cursor — plain Atomic.make *)
}
```

Slot Lifecycle at logical position p

FREE TO WRITE

`sequence = p`

Slot is empty. Producer that wins CAS on enqueue_pos owns it.

→

WRITTEN

`sequence = p+1`

Producer has written value. Consumer can now claim it via CAS.

→

FREE NEXT CYCLE

`sequence = p+C`

Consumer has read value. Producers in the next cycle can write here.

MPMC: try_enqueue — The diff Protocol

```
let try_enqueue t v =
  let size = t.mask + 1 in
  let rec loop () =
    let pos  = Atomic.get t.enqueue_pos in
    let slot = t.buffer.(pos land t.mask) in
    let seq  = Atomic.get slot.sequence in
    let diff = seq - pos in
    if diff = 0 then begin                        (* CASE A: slot is FREE *)
      if Atomic.compare_and_set t.enqueue_pos pos (pos + 1) then begin
        slot.value <- Some v;                     (* write value *)
        Atomic.set slot.sequence (pos + 1);       (* signal consumers *)
        true
      end else loop ()                            (* CAS lost — retry *)
    end else if diff < 0 then begin               (* CASE B: full or in-flight *)
      if Atomic.get t.dequeue_pos > pos - size then begin
        Domain.cpu_relax (); loop ()              (* consumer in-flight — spin *)
      end else false                              (* genuinely full *)
    end else loop ()                              (* CASE C: producer overtook us *)
  in loop ()
```

The three diff cases

diff = 0: seq = pos → slot free for this cycle. Race to claim with CAS.

diff < 0: seq < pos → buffer full OR consumer mid-flight freeing slot.

diff > 0: seq > pos → another producer advanced enqueue_pos past us. Retry.

MPMC: try_dequeue — The Producer In-Flight Spin

```
let try_dequeue t =
  let rec loop () =
    let pos  = Atomic.get t.dequeue_pos in
    let slot = t.buffer.(pos land t.mask) in
    let seq  = Atomic.get slot.sequence in
    let diff = seq - (pos + 1) in
    if diff = 0 then begin                      (* slot WRITTEN for this cycle *)
      if Atomic.compare_and_set t.dequeue_pos pos (pos + 1) then begin
        let v = slot.value in
        slot.value <- None;
        Atomic.set slot.sequence (pos + t.mask + 1); (* free for next cycle *)
        v
      end else loop ()                          (* CAS lost — retry *)
    end else if diff < 0 then begin
      if Atomic.get t.enqueue_pos > pos then begin
        Domain.cpu_relax (); loop ()            (* producer in-flight — MUST SPIN *)
      end else None                             (* truly empty *)
    end else loop ()                            (* another consumer overtook us *)
  in loop ()
```

Critical Linearisability Argument

The linearisation point of a successful try_enqueue is the CAS on enqueue_pos — not the later Atomic.set slot.sequence.
Once the CAS commits, the enqueue is logically complete. Any concurrent try_dequeue at that slot MUST eventually return the value — returning None would violate linearisability.
Therefore: when diff < 0 AND enqueue_pos > pos, we spin. Returning None here would be WRONG.

MPMC: Why sequence = p + mask + 1 Frees the Slot

After a consumer reads slot at position p, it sets:

```
(* mask = capacity - 1, so mask + 1 = capacity *)
Atomic.set slot.sequence (pos + t.mask + 1)
  (* = pos + capacity                          *)
  (* = the sequence value a producer at this slot will see in the NEXT cycle *)
```

Concrete example — capacity = 4 (mask = 3)

Cycle	Slot	Position p	Producer sees seq =	Consumer sets seq =
Cycle 0	[2]	2	2 (free)	3 (written)-> then consumer sets 2+3+1=6
Cycle 1	[2]	6	6 (free for cycle 1)	7 (written)-> consumer sets 6+3+1=10
Cycle 2	[2]	10	10 (free for cycle 2)	11 (written)-> consumer sets 10+3+1=14

The sequence number encodes BOTH the slot's generation AND its state — no separate flag or lock needed.

Initialisation: slot i starts with sequence = i, so the FIRST producer reading slot i sees diff = i - i = 0 (free). No special-casing needed.

SECTION 04

MPSC Queue One-Sided CAS

MPSC: Design Differences from MPMC

Task 3 *Same per-slot sequence technique, but consumer side uses plain Atomic.set (no CAS)*

MPMC (Vyukov)

- Producers: CAS on enqueue_pos
- Consumers: CAS on dequeue_pos
- Both ends are fully contended
- Multiple consumers race for each slot
- dequeue_pos must be CAS'd to prevent two consumers reading the same item

One fewer CAS per operation

MPSC (This Project)

- Producers: CAS on enqueue_pos (same)
- Consumer: plain Atomic.SET on dequeue_pos
- Only one consumer — no race on head
- Eliminates one CAS per dequeue on the hot path
- But adds 'producer in-flight' spin to consumer side

MPSC: Why CAS and Not fetch_and_add?

The project spec requires CAS on the tail. Here is the precise technical reason:

fetch_and_add (FAA) — WRONG here

- FAA is IRREVOCABLE
  - Producer increments cursor without checking if slot is free
  - If buffer is full, producer has already claimed a slot it cannot return
  - Must spin forever until consumer frees the slot
- Breaks non-blocking try_enqueue semantics
  - Cannot return false on a full buffer cleanly
  - try_enqueue that blocks defeats the purpose of a bounded queue

compare_and_set (CAS) — CORRECT

- CAS is REVOCABLE
  - Producer reads pos, checks slot.sequence = pos (free?)
  - Only claims the slot if it is confirmed free
  - CAS failure means another producer claimed it — retry cleanly
- Enables non-blocking try_enqueue
  - If slot is NOT free (diff < 0), return false immediately
  - Producer never commits to a slot it cannot write now

Disruptor MPSC uses FAA because it never needs to return false — producers ALWAYS publish (pipeline model). Our MPSC needs CAS because try_enqueue must be able to return false.

MPSC: try_dequeue — Producer In-Flight Detection

```
let try_dequeue t =
  let pos  = Atomic.get t.dequeue_pos in
  let slot = t.buf.(pos land t.mask) in
  let seq  = Atomic.get slot.sequence in
  let diff = seq - (pos + 1) in
  if diff = 0 then begin                    (* slot WRITTEN — consume it *)
    let v = slot.value in
    slot.value <- None;
    Atomic.set t.dequeue_pos (pos + 1);    (* PLAIN SET — sole consumer *)
    Atomic.set slot.sequence (pos + t.mask + 1);
    v
  end else if diff < 0 then begin
    if Atomic.get t.enqueue_pos > pos then begin
      (* A producer won the CAS but hasn't written value yet.  *)
      (* We MUST spin — returning None would be wrong because   *)
      (* the enqueue has already committed (CAS succeeded).     *)
      let rec spin () =
        Domain.cpu_relax ();
        let seq2 = Atomic.get slot.sequence in
        if seq2 - (pos + 1) = 0 then (* ... read and advance *)
        else spin ()
      in spin ()
    end else None   (* truly empty: enqueue_pos = pos *)
  end else None
```

- Plain Atomic.set on dequeue_pos — no CAS. The sole consumer is the only writer of this field.
- The producer in-flight spin is mandatory: if enqueue_pos > pos, a producer successfully CAS'd and will publish. We cannot return None.

SECTION 05

Disruptor Pipeline
FAA + Available Buffer

Disruptor: Architecture Overview

- **Key components**
- Sequencer — manages the producer cursor (SPSC: plain set; MPSC: FAA)
- RingBuffer — pre-allocated array of slots
- SequenceBarrier — consumer-side poll loop finding highest contiguous published seq
- Sequence.t — padded atomic int (Atomic.make_contended), one per producer/consumer
- **Topology: pipeline, not work-queue**
- EVERY consumer sees EVERY event — fan-out, not fan-in
- This differs from MPMC where each event goes to exactly ONE consumer
- Natural for event-processing pipelines (trading, logging, streaming)
- **Why Disruptor MPMC was not implemented**
- FAA is irrevocable on both producer AND consumer cursors
- Consumer that FAA'd dequeue_pos cannot cancel if producer stalls
- Clean shutdown in nP:nC sweep is inherently unreliable

Disruptor SPSC Sequencer: Cached Consumer Optimisation

```
type t = {
  buffer_size     : int;
  cursor          : Sequence.t;        (* producer's published-up-to, padded *)
  consumer_seq    : Sequence.t;        (* reference to consumer's sequence *)
  mutable cached_consumer : int;       (* local cache of consumer progress *)
}

let next t =
  let next_seq  = Sequence.get t.cursor + 1 in
  let wrap_point = next_seq - t.buffer_size in
  if wrap_point > t.cached_consumer then begin
    (* Slow path: refresh cache and spin until consumer makes progress *)
    let rec spin () =
      let consumer_now = Sequence.get t.consumer_seq in
      t.cached_consumer <- consumer_now;
      if wrap_point > consumer_now then (Domain.cpu_relax (); spin ())
    in spin ()
  end;    (* Fast path: cached value shows space available *)
  next_seq

let publish t sequence = Sequence.set t.cursor sequence
```

The Cached Consumer Trick

On the COMMON (non-full) path, the producer reads its own local integer — zero cross-domain atomic traffic. Only when the buffer appears full does it read the live consumer sequence (an expensive cross-domain atomic). This single optimisation eliminates a cache-coherence round-trip on every hot-path iteration.

Disruptor MPSC Sequencer: FAA Batch Claim + Available Buffer

```
(* FAA: multiple producers each get a unique, non-overlapping range *)
let next t n =
  let hi = Sequence.fetch_and_add t.cursor n + n in  (* atomically claims [lo..hi] *)
  let lo = hi - n + 1 in
  let wrap_point = hi - t.buffer_size in
  if wrap_point > Atomic.get t.cached_gating_min then begin
    let rec spin () =
      let gating_min = min_gating_seq t in
      Atomic.set t.cached_gating_min gating_min;  (* Atomic.t — fixed TSAN race *)
      if wrap_point > gating_min then (Domain.cpu_relax (); spin ())
    in spin ()
  end;
  (lo, hi)

(* Mark each slot published in the available_buffer bitmap *)
let publish t sequence =
  let index = sequence land t.mask in
  let flag  = sequence lsr t.index_shift in
  Atomic.set t.available_buffer.(index) flag
```

Why available_buffer?

- Multiple producers use FAA so they may publish out of order (P2 at seq 5 may publish before P1 at seq 4)
- available_buffer[i] stores the 'cycle' flag for slot i. Consumer barrier scans to find highest CONTIGUOUS published sequence.
- TSAN fix: cached_gating_min changed from mutable int to Atomic.t — multiple producers wrote it concurrently without synchronisation.

Disruptor: Sequence Barrier — Consumer Wait Loop

```
(* Consumer calls wait_for(want_seq) to block until sequence is available *)
let wait_for barrier want_seq =
  let rec spin () =
    let available = highest_published_sequence barrier want_seq in
    if available < want_seq then begin
      Domain.cpu_relax ();
      spin ()   (* keep polling until producer publishes *)
    end else
      available  (* return the highest safe-to-read sequence *)
  in spin ()

(* For MPSC: scan available_buffer to find highest contiguous sequence *)
let highest_published_sequence barrier upper_bound =
  let rec scan seq =
    if seq > upper_bound then seq - 1
    else
      let index = seq land mask in
      let flag  = seq lsr index_shift in
      if Atomic.get available_buffer.(index) = flag
      then scan (seq + 1)
      else seq - 1   (* gap found — stop here *)
  in scan (consumer_seq + 1)
```

Consumer processes events from consumer_seq+1 up to the returned available sequence in a tight batch loop — no per-item barrier calls needed.

SECTION 06

Testing & Verification

Testing Strategy — Four Layers

L1

Smoke Tests

test_spsc_smoke.ml — Basic enqueue/dequeue ordering, full-buffer rejection, empty-queue None under sequential execution.

L2

Stress Tests

test_spsc_stress / mpsc_stress / mpmc_stress — Concurrent domains push 1M items each. Verify no loss or duplication by collecting all items and checking the union.

L3

QCheck-STM

agree_test (sequential) and agree_test_par (parallel) — Random command sequences checked against a sequential model. Finds interleaving bugs.

L4

QCheck-Lin

lin_test — Linearisability checker. Verifies every concurrent execution has a valid sequential interleaving consistent with the model.

All 9 STM/Lin tests pass. All 11 comparison suite tests pass.

QCheck-STM: The Sequential Model

All three queues are tested against the same sequential specification:

```
let cap = 8

type state = { contents : int list; capacity : int }
let init_state = { contents = []; capacity = cap }

type cmd = Enqueue of int | Dequeue

let next_state cmd st = match cmd with
  | Enqueue v ->
    if List.length st.contents >= st.capacity then st   (* full — no change *)
    else { st with contents = st.contents @ [v] }
  | Dequeue ->
    (match st.contents with
     | []     -> st                                     (* empty — no change *)
     | _ :: t -> { st with contents = t })

let run_cmd cmd (sut : Queue_spsc.t) = match cmd with
  | Enqueue v -> Res (bool, Queue_spsc.try_enqueue sut v)
  | Dequeue   -> Res (option int, Queue_spsc.try_dequeue sut)
```

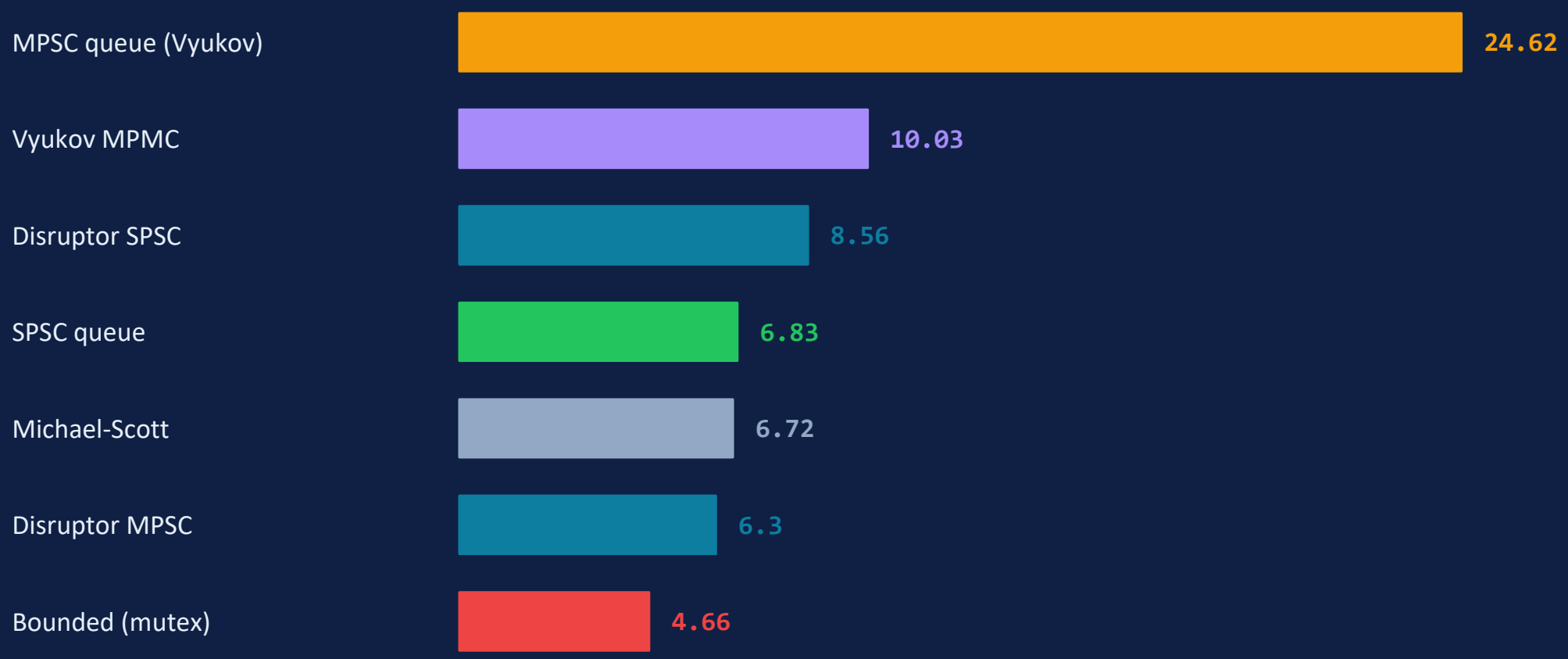
SPSC Adapter: wraps each side in a per-side mutex for the test harness (the queue itself is unchanged — only the test scaffolding is serialised).

SECTION 07

Benchmarks & Results

Benchmark: 1P:1C Baseline (M ops/sec)

2 seconds per run · 65536-slot buffer · higher is better



Key insight: MPSC at 1P:1C is fastest because its dequeue path has NO CAS. Slot sequence numbers are cheaper than option-boxing. SPSC is not the winner here despite simplest design.

Benchmark: nP:1C — The MPSC Regime

Disruptor MPSC scales UP with more producers. Our CAS-based MPSC collapses.

	n=1	n=2	n=3	n=4	n=5	n=6	n=7	n=8
Disruptor MPSC	7.1	7.4	9.69	10.21	10.1	12.59	12.42	12.54
MPSC queue	25.83	5.23	3.25	2.38	2	2	1.91	2
Vyukov MPMC	7.83	3.57	2.53	2.35	2.16	1.87	1.76	1.22
Michael-Scott	5.96	3.76	2.98	2.47	2.15	2.04	1.79	1.81
Bounded(mutex)	3.88	3.32	2.23	2.04	1.7	1.66	1.41	1.33

FAA (Disruptor MPSC) does not retry — each producer's increment is unconditional. More producers = more total throughput. CAS-based MPSC suffers retry storms.

Batch-Size Sweep: Disruptor SPSC Peak at k=4

1P:1C · Batch k = items per producer burst / consumer drain

	k=1	k=4	k=16	k=64	k=256
Disruptor SPSC	6.09	**32.04**	7.3	7.31	7.71
Vyukov MPMC	6.62	8.17	7.61	8.53	**20.71**
Disruptor MPSC	6.79	6.82	6.37	6.55	6.44
Michael-Scott	5.08	2.74	4.29	5.43	6.34
Bounded(mutex)	4	3.62	3.95	3.71	3.69

32.04M at k=4: consumer drains all queued events per barrier pass, amortising barrier scan cost. Effect vanishes at k=16+ as per-item cost re-dominates.

False Sharing: Atomic.make_contended vs Atomic.make

Padded (Atomic.make_contended) vs Unpadded (Atomic.make) on indices

Implementation	Config	Padded	Unpadded	Delta%
Disruptor SPSC	1P:1C	7.2M	7.36M	-2.2%
Disruptor MPSC	1P:1C	5.56M	4.97M	+11.9%
Disruptor MPSC	4P:1C	8.37M	1.81M	+362.7%
Vyukov MPMC	1P:1C	5.8M	7.54M	-23.1%
Vyukov MPMC	2P:1C	3.83M	2.73M	+40.1%
Vyukov MPMC	4P:4C	2.24M	1.4M	+59.9%

+362.7% at Disruptor MPSC 4P:1C — the headline result. 4 producers all writing available_buffer near the same cursor line causes a cache-coherence storm without padding.

False Sharing: Why It Happens & When Padding Hurts

False Sharing — the problem

- CPUs transfer memory in 64-byte cache lines
- If head & tail share a cache line:
  - Producer writes tail -> invalidates consumer's cached line
  - Consumer writes head -> invalidates producer's cached line
- Every operation causes a cross-core coherence round-trip
- 10x throughput reduction in worst case

Padding — the fix

- Atomic.make_contended: OCaml 5 runtime pads the value to its own 64-byte line
- head and tail each live on separate cache lines
- Producer writes tail -> only invalidates producer's line
- No cross-domain false invalidation

When does padding HURT?

At 1P:1C, the producer and consumer never actually write the SAME atomic variable. Padding spreads the indices across more cache lines than necessary, reducing spatial locality. Vyukov MPMC shows -23% at 1P:1C because its unpadded enqueue/dequeue positions happen to sit on a warm shared line that the single producer and consumer access sequentially. The breakeven is at 2P:1C where inter-domain contention begins dominating.

SECTION 08

Conclusions

Answering the Research Question

Ring buffers gain 1.3x-3.7x over Michael-Scott at 1P:1C. The advantage shrinks at 4P:4C (Vyukov MPMC leads by only ~1.03x) as symmetric CAS contention erases the allocation savings.

Design Space Trade-offs

SPSC -> MPSC	Large performance continuity. MPSC with 1P often BEATS SPSC because per-slot sequences + no CAS on consumer path > plain Option array.
MPSC -> MPMC	MOST DRAMATIC INFLECTION. Adding CAS on consumer side doubles atomic traffic. At 4P:4C the Vyukov MPMC advantage over MS queue is nearly zero.
CAS vs FAA	FAA (Disruptor) scales UP with producer count to 12.59M at 6P. CAS (our MPSC) collapses to 2M at 5P. FAA irrevocability is the fundamental trade-off.
False sharing	+362% at 4P:1C with padding. Cache-line discipline is not an optimisation — it is a correctness-adjacent concern for any multi-core deployment.

Thank You

Bounded Lock-Free Queues: From SPSC to MPMC

3 queues	SPSC · MPSC (CAS) · MPMC (Vyukov)
+ Disruptor	SPSC & MPSC pipeline variants
Testing	Smoke · Stress · QCheck-STM · QCheck-Lin — all pass
Key result	+362% from cache-line padding at 4P:1C
Inflection	MPSC -> MPMC: biggest complexity-performance shift